

Dynamic Semantics and DSL Introduction

Jibesh Patra, Soumyajit Dey

Week 7 – Software Engineering – CS20202 – Spring 2026 – IIT Kharagpur

Adpated from: Crafting Interpreters by Robert Nystrom

Dynamic Semantics

- Defines the effect, the **execution** of phrases should have.
- The *abstract expressions* should produce a value or diverge and lead to an erroneous situation.
- **Environment Model** for dynamic semantics:
 - Expressions are evaluated in an environment binding variables to values.
 - Functions appear as values called **closures**.

Closure Example (Python)

```
1 def get_multiple(n):
2     def multiple(x):
3         return x * n
4     return multiple
5
6 mul3 = get_multiple(3)
7 mul5 = get_multiple(5)
8
9 print(mul3(20))
10 print(mul3(40))
11 print(mul5(20))
```

- Here, a function `get_multiple` defines an inner function `multiple` and returns it.
- The inner function is called **closure**.
- The `mul3` and `mul5` remembers the value of `n` even after the call `get_multiple` has finished.

- A lambda expression $\lambda x.e$ evaluated in a value environment V results in the closure (x, e, V) . The argument is x , the expression is e and the value environment is V provides values for the free variables of $\lambda x.e$.
- Closures for recursive functions: (f, x, e, V) . The name of the function is f , argument variable is x & e is the body of the function. The value environment V provides values for free variables of e except f and x . The bindings for f & x are added to V when the recursive closure is applied.

Closure Example (OCaml)

```
1 let a = 3  
2 let f = fun x → x + a
```

- Conceptually results in the closure $f = (x, x + a, a \rightarrow 3)$.
- Calling `f 5` results in $5 + 3 = 8$.

$$V \vdash e \triangleright v$$

- The above is the general form of **evaluation judgement**.
- Given the value environment V , the expression e evaluates to v . In a value environment, there exists a finite collection of bindings $x \triangleright v$ mapping variables to values.
- The possible values for v are:
 - numbers
 - booleans
 - strings
 - tuples of values
 - closures

- Assume constants c are values (numbers, booleans, ...).
- For operators $v_1 \text{ op } v_2 = v$, the assumption is made that successful operator application results in v .
- An expression e is **evaluable** in an environment V if there is a value v such that the judgement $V \vdash e \triangleright v$ is derivable.

Typing Rules (1/3): Constants and Conditionals

$$\frac{}{V \vdash c \triangleright c} \quad \frac{(x \triangleright v) \in V}{V \vdash x \triangleright v}$$

$$\frac{V \vdash e_1 \triangleright v_1 \quad V \vdash e_2 \triangleright v_2 \quad v_1 \circ v_2 = v}{V \vdash e_1 \circ e_2 \triangleright v}$$

$$\frac{V \vdash e_1 \triangleright \mathbf{true} \quad V \vdash e_2 \triangleright v}{V \vdash \mathbf{IF} \ e_1 \ \mathbf{THEN} \ e_2 \ \mathbf{ELSE} \ e_3 \triangleright v}$$

$$\frac{V \vdash e_1 \triangleright \mathbf{false} \quad V \vdash e_3 \triangleright v}{V \vdash \mathbf{IF} \ e_1 \ \mathbf{THEN} \ e_2 \ \mathbf{ELSE} \ e_3 \triangleright v}$$

Typing Rules (2/3): Tuples and Functions

$$\frac{V \vdash e_1 \triangleright v_1 \quad \dots \quad V \vdash e_n \triangleright v_n}{V \vdash (e_1, \dots, e_n) \triangleright (v_1, \dots, v_n)}$$
$$\frac{V \vdash e \triangleright (v_1, \dots, v_n) \quad 1 \leq i \leq n}{V \vdash \pi_i e \triangleright v_i}$$
$$\frac{V \vdash e_1 \triangleright v_1 \quad V, x \triangleright v_1 \vdash e_2 \triangleright v}{V \vdash \text{let } x = e_1 \text{ in } e_2 \triangleright v}$$

Typing Rules (3/3): Recursion

$$\frac{\frac{V \vdash \lambda x. e \triangleright (x, e, V)}{V \vdash e_1 \triangleright (x, e, V') \quad V \vdash e_2 \triangleright v_2 \quad V', x \triangleright v_2 \vdash e \triangleright v}}{V \vdash e_1 \ e_2 \triangleright v} \quad \frac{V, f \triangleright (f, x, e_1, V) \vdash e_2 \triangleright v}{V \vdash \text{LET REC } f x = e_1 \text{ IN } e_2 \triangleright v}$$
$$\frac{V \vdash e_1 \triangleright v_1 \quad V \vdash e_2 \triangleright v_2 \quad v_1 = (f, x, e, V') \quad V', f \triangleright v_1, x \triangleright v_2 \vdash e \triangleright v}{V \vdash e_1 \ e_2 \triangleright v}$$

Example Derivation

$$\frac{\frac{}{\Box \vdash \lambda x. \lambda y. x \triangleright (x, \lambda y. x, \Box)} \quad \frac{}{\Box \vdash 1 \triangleright 1} \quad \frac{}{x \triangleright 1 \vdash \lambda y. x \triangleright (y, x, [x \triangleright 1])}}{\Box \vdash (\lambda x. \lambda y. x) 1 \triangleright (y, x, [x \triangleright 1])}$$

- The evaluation rules do not contain any types which allows the execution of **ill-typed** expressions.
- Execution of ill-typed expressions can lead to erroneous situations.
- With **type safety**, only well-typed expressions are evaluated.
- OCaml is a type safe language.

Domain Specific Language

What is a Domain Specific Language (DSL)?

- We have seen general-purpose languages (GPLs) such as Java, C++, Python.
- A language with a higher level of abstraction optimized for a specific class of problems. A DSL uses the concepts and rules from the field or domain.
- A DSL uses the concepts and rules from the field or domain while sacrificing some of the flexibility to express any program.
- Examples:
 - [Mermaid JS](#)
 - [Jinja](#)
 - Google sheets
 - JUnit

Why?

- Someone does not know how to program.
- Does not want to use ChatGPT or similar tools.

Game – Example 1

 prince_crusher.jp

```
1  khel 800 600 "The Prince Crusher"
```

Game – Example 1

 prince_crusher.jp

```
1  khel 800 600 "The Prince Crusher"  
2  
3  aaaa prince "prince.png" 400 200  
4  aaaa princess "princess.png" 500 450  
5  aaaa tree "tree.png" 10 200  
6  aaaa monster "monster.png" 400 300
```

Game – Example 1

 prince_crusher.jp

```
1  khel 800 600 "The Prince Crusher"
2
3  aaaa prince "prince.png" 400 200
4  aaaa princess "princess.png" 500 450
5  aaaa tree "tree.png" 10 200
6  aaaa monster "monster.png" 400 300
7
8  var move 10
9  var monster_dir 1
10 var monster_speed 20
11
12 on_key "up" { set prince.y (prince.y + move) }
```

Game – Example 1

 prince_crusher.jp

```
1  khel 800 600 "The Prince Crusher"
2
3  aaoo prince "prince.png" 400 200
4  aaoo princess "princess.png" 500 450
5  aaoo tree "tree.png" 10 200
6  aaoo monster "monster.png" 400 300
7
8  var move 10
9  var monster_dir 1
10 var monster_speed 20
11
12 on_key "up" { set prince.y (prince.y + move) }
13 on_key "down" { set prince.y (prince.y - move) }
14 on_key "left" { set prince.x (prince.x - move) }
15 on_key "right" { set prince.x (prince.x + move) }
```



Game – Example 1

```
16 loop {  
17     set monster.x (monster.x + (monster_speed * monster_dir)  
    )
```

Game – Example 1

```
16 loop {
17     set monster.x (monster.x + (monster_speed * monster_dir)
18         )
19     if (monster.x > 750) { set monster_dir (0 - 1) }
20     if (monster.x < 200) { set monster_dir 1 }
21
22     if (tokkor prince monster) { tata }
23 }
```

The game gets compiled to  Python and uses the arcade library.

Lexing



What is Scanning?

- Take raw source code as a string and translate the string into a list of tokens representing words.

`aaoo prince "prince.png" 400 200`

- The word `aaoo` has a meaning but not `aa`.



Example of Tokens

Keywords

```
khe1  
aaoo  
var  
on_key  
loop  
set  
if  
tata  
tokkor
```

Brackets and Operators

```
{ ( . + * - > <
```

Identifier & Literals

```
monster, prince, ...  
"up", "down", ...  
1, 20, ...
```

Mapping Token to Kind of Token

Token	Kind
khel	KHEL
aaoo	AAOO
var	VAR
on_key	ON_KEY
loop	LOOP
set	SET
if	IF
tata	TATA
tokkor	TOKKOR

Token	Kind
{	LEFT_CURL
}	RIGHT_CURL
(	LEFT_PAR
)	RIGHT_PAR
.	DOT
+	PLUS
*	STAR
-	MINUS
<	LESS
>	MORE

Token Example	Kind
monster	IDF
"prince.png"	STR
20	INT

Token	Kind
program end	END

Create OCaml Project

- dune is build system for OCaml projects.
- Create dune project using `dune init proj my-dsl`
- `cd my-dsl`
- Execute `dune build` and `dune exec bin/main.exe`. Change the code and continuously execute the project using this command.

Steps of Compilation

 main.ml

```
1 let () =  
2   let tokens = tokenize (explode source_game) in  
3   let prog = parse tokens in  
4   static_dynamic_checks prog;  
5   generate_py_file prog "./generated-games/prince_crusher.py"
```

Test-driven Development (TDD)

1. Read the code game file  `prince_crusher.jp` and print to the console.

Test-driven Development (TDD)

1. Read the code game file  `prince_crusher.jp` and print to the console.
2. TDD: We write tests and write code to satisfy the tests. Instead of working on the entire game code, let us write a small test.
3. We will use `JUnit` along with printing to the console during development. `JUnit` primarily for testing individual functions.

Input:

 `prince_crusher.jp`

```
1 {}
```

Test-driven Development (TDD)

1. Read the code game file  `prince_crusher.jp` and print to the console.
2. TDD: We write tests and write code to satisfy the tests. Instead of working on the entire game code, let us write a small test.
3. We will use `JUnit` along with printing to the console during development. `JUnit` primarily for testing individual functions.

Input:

 `prince_crusher.jp`

1 `{}`

Output:

```
LEFT_CURL { null
RIGHT_CURL } null
END null
```

Scanning More Characters & Lexical Errors

Input:

 prince_crusher.jp

1 ({}).+-

Output:

```
LEFT_PAR ( null
LEFT_CURL { null
RIGHT_CURL } null
RIGHT_PAR ) null
DOT . null
PLUS + null
MINUS - null
```


Scanning More Characters & Lexical Errors

Input:

 prince_crusher.jp

1 ({}).+-

Output:

```
LEFT_PAR ( null
LEFT_CURL { null
RIGHT_CURL } null
RIGHT_PAR ) null
DOT . null
PLUS + null
MINUS - null
```

Input:

 prince_crusher.jp

1 ?(|{}).+-

Output:

```
Lexing Error Illegal Character '?'
Lexing Error Illegal Character '|'
LEFT_PAR ( null
LEFT_CURL { null
RIGHT_CURL } null
RIGHT_PAR ) null
DOT . null
PLUS + null
MINUS - null
```

More Examples (1)

Input:

 prince_crusher.jp

```
1 (      )  
2  
3 { }
```

Output:

```
LEFT_PAR ( null  
RIGHT_PAR ) null  
LEFT_CURL { null  
RIGHT_CURL } null  
END null
```

More Examples (1)

Input:

 prince_crusher.jp

```
1 (      )  
2  
3 { }
```

Output:

```
LEFT_PAR ( null  
RIGHT_PAR ) null  
LEFT_CURL { null  
RIGHT_CURL } null  
END null
```

More Examples (2)

Input:

 prince_crusher.jp

```
1 khel 800 600 "The Prince Crusher"  
2 aaoo prince "prince.png" 400 200  
3 var move 10  
4 on_key "up" { set prince.y (prince.y + move) }
```

Output:

```
INT 800 800  
INT 600 600  
STR "The Prince Crusher" The Prince Crusher  
AA00 aao null  
IDF prince null  
STR "prince.png" prince.png  
INT 400 400  
INT 200 200  
VAR var null
```

More Examples (2)

Input:

 prince_crusher.jp

```
1 khel 800 600 "The Prince Crusher"  
2 aaoo prince "prince.png" 400 200  
3 var move 10  
4 on_key "up" { set prince.y (prince.y + move) }
```

Output:

```
INT 800 800  
INT 600 600  
STR "The Prince Crusher" The Prince Crusher  
AAOO aao null  
IDF prince null  
STR "prince.png" prince.png  
INT 400 400  
INT 200 200  
VAR var null
```

```
IDF move null  
INT 10 10  
ON_KEY on_key null  
STR "up" up  
LEFT_CURL { null  
SET set null  
IDF prince null  
DOT . null  
IDF y null  
LEFT_PAR ( null  
IDF prince null  
DOT . null  
IDF y null  
PLUS + null  
IDF move null  
RIGHT_PAR ) null  
RIGHT_CURL } null  
END null
```

Write a **Tokenizer** for the entire game.



This is the End

